

Building the RGB Knowledge Base

How I turned a research paper into something you can query

Author: Amir Mahdi Zarei Nejad

Live demo: <https://rag-chatbot-mauve-nine.vercel.app/>

Date: June 2026

What this is

The assignment is the **knowledge base**, not the chat UI. I took one paper, *Benchmarking Large Language Models in Retrieval-Augmented Generation* (Chen et al., 2024), cleaned it up, split it into chunks, and wired up search so you can ask questions and see which passages match.

The web app is just a way to poke at retrieval. The real work is: pick the source, ingest the text, chunk it, rank it.

The paper (my source)

RGB is a benchmark for testing LLMs under RAG. The paper defines four skills models need:

- **Noise robustness** - pull the answer out when docs have extra noise
- **Negative rejection** - say "I don't know" when nothing relevant was retrieved
- **Information integration** - combine facts from multiple docs
- **Counterfactual robustness** - spot wrong info in retrieved docs

They ran six models through it (ChatGPT, ChatGLM-6B, ChatGLM2-6B, Vicuna-7B, Qwen-7B-Chat, BELLE-7B). Bottom line: models handle some noise, but still fail on rejection, multi-doc answers, and bad context.

How it works

```
User question
  |
  v
read paper text from disk
  |
  v
split into paragraph chunks (chunk-1, chunk-2, ...)
  |
  v
MiniSearch ranks chunks, return top 3
  |
  v
show chunks in UI (+ optional LLM answer)
```

Stack: Next.js API route, `lib/retrieval/`, MiniSearch (local TF-IDF). No DB, no embeddings API.

File	Does what
<code>lib/retrieval/chunk.ts</code>	Splits text on blank lines into chunks
<code>lib/retrieval/search.ts</code>	Indexes chunks, searches, normalizes scores
<code>app/api/chat/route.ts</code>	Runs retrieval, skips LLM if nothing matches

What I built (in order)

1. **Scaffold** - Next.js + TypeScript project
2. **Retrieval pipeline** - chunking, MiniSearch, API route
3. **Demo UI** - ask a question, see matched chunks in a sidebar
4. **Swapped the source** - replaced placeholder Persian notes with the full RGB paper

Biggest pivot: I started with a short Persian RAGAS summary as a placeholder, then replaced it with the actual RGB paper once the retrieval stack worked.

Code that matters

Chunking. One blank line between paragraphs = one chunk.

```
export function createKnowledgeChunks(knowledgeBase: string) {
  return knowledgeBase
    .split(/\n\s*\n/)
    .map((p) => p.trim())
    .filter(Boolean)
    .map((text, i) => ({ id: `chunk-${i + 1}`, text }))
}
```

Search. Build index, search question, return top 3 with scores normalized to 0-1.

```
export function retrieveFromText(question, knowledgeBase, options = {}) {
  const maxChunks = options.maxChunks ?? 3
  const chunks = createKnowledgeChunks(knowledgeBase)
  const search = createSearchIndex(chunks)
  const results = search.search(question, { prefix: true })
  const topScore = results[0]?.score ?? 0

  return results.slice(0, maxChunks).map((r) => ({
    id: String(r.id),
    score: topScore > 0 ? r.score / topScore : 0,
    text: String(r.text ?? ''),
  }))
}
```

API short-circuit. No chunks found = no LLM call, just return early.

```
const chunks = retrieveRelevantChunks(body.question)

if (chunks.length === 0) {
  return NextResponse.json({
    answer: "The retrieved context is not sufficient to answer this question.",
    chunks,
    model: "local-retrieval",
  })
}
```

Try it

Live: rag-chatbot-mauve-nine.vercel.app

Good test questions:

- What abilities does RGB evaluate?
- Which LLMs were evaluated on RGB?
- What is noise robustness in RAG?

Wrap-up

I built a queryable copy of the RGB paper: ingest, clean, chunk, search. The demo shows exactly which passages get picked for each question. That's the assignment.